Experiment No. : 7

Title: Clean, Validate and Store Scraped/API Data in CSV for Analysis

Batch: b3 **Roll No.: 16014225801** **Experiment No.: 5**

Aim

To clean and validate data obtained through web scraping or APIs and store the processed data in **CSV format** for further analysis.

Resources Required

- Python 3.x
- Selenium
- VS Code / PyCharm / Jupyter Notebook
- Web browser – Chrome/Edge/Firefox

Theory:

Data obtained through web scraping and APIs is often not immediately ready for analysis. For example, scraped/API data may contain:

Missing values
Duplicate records
Extra spaces
Different text formats
Currency symbols
Invalid values
Incorrect data types
Unwanted characters

Therefore, a typical data-processing workflow is:

Web Scraping / API
↓
Raw Data
↓
Validate
↓
Clean
↓
Verify
↓
Store as CSV
↓
Data Analysis

1. Data Validation

Data validation checks whether the acquired data satisfies expected conditions.
For example, if product ratings are expected to be between 1 and 5:

Rating = 4.5 ✓ Valid
Rating = 3.8 ✓ Valid

Rating = 7.2 ✗ Invalid

Common validation checks include:

- Data type
- Range
- Required fields
- Format
- Missing values
- Duplicate records



2. Data Cleaning

Data cleaning involves identifying and appropriately handling problematic data.

Example

Raw data:

Product	Price	Rating
Laptop A	₹55,999	4.5
laptop a	₹55,999	4.5
Mouse	₹800	4.2
Keyboard	N/A	4.1

Possible problems:

- Different capitalization of product names
- Currency symbol in price
- Missing price
- Possible duplicate product

After cleaning:

Product	Price	Rating
Laptop A	55999	4.5
Mouse	800	4.2
Keyboard	Missing	4.1

3. Handling Missing Values

Missing values can occur because:

- The website does not provide the information.
- The API returns a null value.
- Scraping fails to identify an element.
- The source itself contains incomplete information.

Example:

Product	Price
Laptop	55000
Mouse	800
Keyboard	NaN

Depending on the application, missing values may be:

- retained,
- replaced,

- or removed.

Students should not automatically replace every missing value with zero.

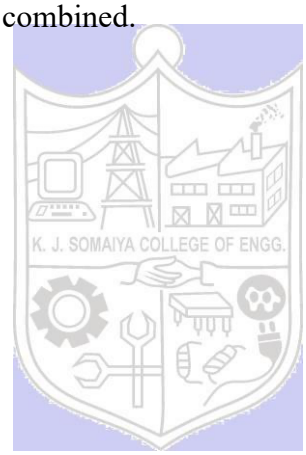
4. Handling Duplicate Records

Duplicate records may occur when:

- The same webpage is scraped more than once.
- API pages overlap.
- The same record is returned repeatedly.
- Data from multiple sources is combined.

Example:

P101
P102
P101
P103
P102
After duplicate removal:
P101
P102
P103



5. Standardizing Data

The same value may appear in different forms.

Example:

Mumbai
mumbai
MUMBAI
Mumbai

These can be standardized to:

Mumbai

Similarly:

₹55,999
Rs. 55,999
55999

can be converted into a consistent numerical representation where appropriate.

6. CSV Format

CSV (Comma-Separated Values) is a simple tabular data format.

Example:

Product,Price,Rating
Laptop A,55999,4.5
Mouse,800,4.2
Keyboard,1500,4.1

CSV is useful because it can be opened using:

- Microsoft Excel
- Google Sheets
- Python
- Pandas

- Other data-analysis tools

Software / Requirements

- Python 3.x
- Google Colab / Jupyter Notebook / Python IDE
- Pandas
- Data obtained from a scraper or API

Install Pandas if required:

```
!pip install pandas
```

Import:

```
import pandas as pd
```

Procedure

Step 1 — Prepare the raw data

For demonstration, we create a dataset representing data obtained from a website/API. import pandas as pd. Consider Following example:

```
data = {
    "Product": [
        "Laptop A",
        "laptop a",
        "Mouse",
        "Keyboard",
        "Mouse"
    ],
    "Price": [
        "₹55,999",
        "₹55,999",
        "₹800",
        "N/A",
        "₹800"
    ],
    "Rating": [
        4.5,
        4.5,
        4.2,
        4.1,
        4.2
    ]
}

df = pd.DataFrame(data)

print(df)
```

Step 2 — Inspect the data

Step 3 — Check missing values

Step 4 — Convert "N/A" into a missing value

Step 5 — Clean the Price Column

Step 6 — Standardize Product Names
Convert product names into a consistent format:

For example:
laptop a
becomes:
Laptop A

Step 7 — Identify Duplicate Records and Remove duplicate rows

Step 8 — Validate the Rating
Suppose the acceptable rating range is:
1 to 5
We can check:

```
invalid_rating = df[
   (df["Rating"] < 1) |
   (df["Rating"] > 5)
]

print(invalid_rating)
```

Step 9 — Validate Required Fields
Suppose Product is a mandatory field.

```
print(df["Product"].isnull().sum())
```

If required fields contain missing values, they should be handled according to the application's requirements.

Step 10 — Inspect the Cleaned Data

```
print(df)
```

A possible cleaned dataset is:

Product	Price	Rating
Laptop A	55999	4.5
Mouse	800	4.2
Keyboard	NaN	4.1

Step 11 — Store Data in CSV

Step 12 — Verify the CSV File
Read the CSV file again:

cleaned_df = pd.read_csv("cleaned_product_data.csv")

print(cleaned_df)
This confirms that the cleaned data has been successfully stored.

Expected Output

Raw Data

	Product	Price	Rating
0	Laptop A	₹55,999	4.5
1	laptop a	₹55,999	4.5
2	Mouse	₹800	4.2
3	Keyboard	N/A	4.1
4	Mouse	₹800	4.2

Cleaned Data

	Product	Price	Rating
0	Laptop A	55999.0	4.5
2	Mouse	800.0	4.2
3	Keyboard	NaN	4.1

CSV File
cleaned_product_data.csv

Observation Table

Sr. No.	Operation	Observation
1	Raw data inspection	Identified inconsistent and duplicate records
2	Missing-value check	Missing values identified
3	Price cleaning	Currency symbol and commas removed
4	Data-type conversion	Price converted to numeric
5	Text standardization	Product names standardized
6	Duplicate removal	Duplicate records removed
7	Validation	Rating range checked
8	CSV storage	Cleaned data stored successfully
9	Verification	CSV file read successfully

Program and Output with screenshots:

```
import pandas as pd

# ------------------------------------------------------------
# Step 1: Prepare Raw Data
# ------------------------------------------------------------

data = {
    "Product": [
        "Laptop A",
        "laptop a",
```

```
        "Mouse",
        "Keyboard",
        "Mouse"
    ],
    "Price": [
        "₹55,999",
        "₹55,999",
        "₹800",
        "N/A",
        "₹800"
    ],
    "Rating": [
        4.5,
        4.5,
        4.2,
        4.1,
        4.2
    ]
}

df = pd.DataFrame(data)

print("========== RAW DATA ==========")
print(df)


# ------------------------------------------------------------
# Step 2: Inspect the Data
# ------------------------------------------------------------

print("\n========== DATA INFORMATION ==========")
print(df.info())


# ------------------------------------------------------------
# Step 3: Check Missing Values
# ------------------------------------------------------------

print("\n========== MISSING VALUES ==========")
print(df.isnull().sum())


# ------------------------------------------------------------
# Step 4: Convert N/A into Missing Value
# ------------------------------------------------------------

df["Price"] = df["Price"].replace("N/A", pd.NA)
```

```
print("\n========== AFTER CONVERTING N/A ==========")
print(df)


# ------------------------------------------------------------
# Step 5: Clean Price Column
# Remove ₹ and commas and convert to numeric
# ------------------------------------------------------------

df["Price"] = (
    df["Price"]
    .astype("string")
    .str.replace("₹", "", regex=False)
    .str.replace(",", "", regex=False)
)

df["Price"] = pd.to_numeric(df["Price"], errors="coerce")

print("\n========== CLEANED PRICE COLUMN ==========")
print(df)


# ------------------------------------------------------------
# Step 6: Standardize Product Names
# ------------------------------------------------------------

df["Product"] = df["Product"].str.strip().str.title()

print("\n========== STANDARDIZED PRODUCT NAMES ==========")
print(df)


# ------------------------------------------------------------
# Step 7: Identify and Remove Duplicate Records
# ------------------------------------------------------------

print("\n========== DUPLICATE RECORDS ==========")
print(df[df.duplicated(keep=False)])

df = df.drop_duplicates().reset_index(drop=True)

print("\n========== AFTER DUPLICATE REMOVAL ==========")
print(df)


# ------------------------------------------------------------
# Step 8: Validate Rating
# Acceptable range: 1 to 5
```

```
# ------------------------------------------------------------

invalid_rating = df[
    (df["Rating"] < 1) |
    (df["Rating"] > 5)
]

print("\n========== INVALID RATINGS ==========")

if invalid_rating.empty:
    print("No invalid ratings found.")
else:
    print(invalid_rating)


# ------------------------------------------------------------
# Step 9: Validate Required Field
# Product is mandatory
# ------------------------------------------------------------

print("\n========== REQUIRED FIELD VALIDATION ==========")

missing_products = df["Product"].isnull().sum()

print("Missing Product values:", missing_products)

if missing_products == 0:
    print("Product field validation successful.")
else:
    print("Missing Product values found.")


# ------------------------------------------------------------
# Step 10: Inspect Cleaned Data
# ------------------------------------------------------------

print("\n========== FINAL CLEANED DATA ==========")
print(df)


# ------------------------------------------------------------
# Step 11: Store Data in CSV
# ------------------------------------------------------------

file_name = "cleaned_product_data.csv"

df.to_csv(file_name, index=False)
```

```
print("\n========== CSV STORAGE ==========")
print("Data successfully stored in:", file_name)


# ------------------------------------------------------------
# Step 12: Verify CSV File
# ------------------------------------------------------------

cleaned_df = pd.read_csv(file_name)

print("\n========== VERIFIED CSV DATA ==========")
print(cleaned_df)

print("\nCSV verification successful.")
```

Output:

```
========== RAW DATA ==========
    Product    Price  Rating
0  Laptop A  ₹55,999     4.5
1  laptop a  ₹55,999     4.5
2     Mouse     ₹800     4.2
3  Keyboard      N/A     4.1
4     Mouse     ₹800     4.2

========== DATA INFORMATION ==========
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Product  5 non-null      str
 1   Price    5 non-null      str
 2   Rating   5 non-null      float64
dtypes: float64(1), str(2)
memory usage: 319.0 bytes
None

========== MISSING VALUES ==========
Product    0
Price      0
Rating     0
dtype: int64

========== AFTER CONVERTING N/A ==========
    Product    Price  Rating
0  Laptop A  ₹55,999     4.5
1  laptop a  ₹55,999     4.5
2     Mouse     ₹800     4.2
```

```
3  Keyboard      NaN     4.1
4     Mouse     ₹800     4.2

========== CLEANED PRICE COLUMN ==========
    Product  Price  Rating
0  Laptop A  55999     4.5
1  laptop a  55999     4.5
2     Mouse    800     4.2
3  Keyboard   <NA>     4.1
4     Mouse    800     4.2

========== STANDARDIZED PRODUCT NAMES ==========
    Product  Price  Rating
0  Laptop A  55999     4.5
1  Laptop A  55999     4.5
2     Mouse    800     4.2
3  Keyboard   <NA>     4.1
4     Mouse    800     4.2

========== DUPLICATE RECORDS ==========
    Product  Price  Rating
0  Laptop A  55999     4.5
1  Laptop A  55999     4.5
2     Mouse    800     4.2
4     Mouse    800     4.2

========== AFTER DUPLICATE REMOVAL ==========
    Product  Price  Rating
0  Laptop A  55999     4.5
1     Mouse    800     4.2
2  Keyboard   <NA>     4.1

========== INVALID RATINGS ==========
No invalid ratings found.

========== REQUIRED FIELD VALIDATION ==========
Missing Product values: 0
Product field validation successful.

========== FINAL CLEANED DATA ==========
    Product  Price  Rating
0  Laptop A  55999     4.5
1     Mouse    800     4.2
2  Keyboard   <NA>     4.1
```

Result

Thus, the scraped/API data was inspected, validated and cleaned by handling missing values, duplicate records, inconsistent formats and data types. The resulting dataset was successfully stored in CSV format for further analysis.

AI Use-Case Mapping(Post Lab Questions)

1. Climate Intelligence for Heatwave Monitoring, Prediction and Early Warning

Post-lab question:

Weather data obtained through an API contains missing temperature values, duplicate observations and inconsistent date formats. Explain how you would validate and clean the dataset before storing it in CSV for use in an AI-based heatwave monitoring and prediction system.

Answer:

Weather data obtained through an API should first be validated and cleaned before being stored in CSV for an AI-based heatwave prediction system. Missing temperature values should be identified and either interpolated, replaced using appropriate statistical methods, or removed depending on their extent. Duplicate observations should be detected using fields such as date, time, and location and removed. Inconsistent date formats should be converted into a standard format such as YYYY-MM-DD. Temperature values should be converted to numeric data types and checked for realistic ranges. Other missing or invalid values should also be handled appropriately. Finally, the cleaned dataset should be verified and stored in CSV format for use in machine-learning models for heatwave monitoring, prediction, and early warning.

Conclusion

Thus, the scraped data was successfully validated, cleaned, standardized, and stored in CSV format. Missing values, duplicate records, inconsistent product names, currency symbols, and data types were handled appropriately. The final verified dataset is now suitable for further analysis and AI-based applications.